

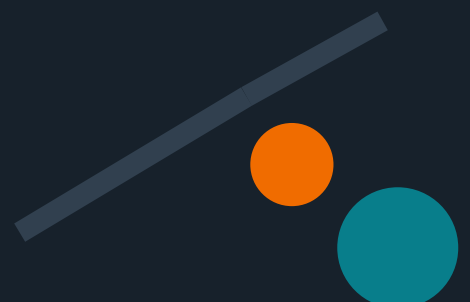
PROYECTO FINAL IMT-342

Guía práctica del Hito 1 ROS 2, URDF/Xacro y cinemática del ABB IRB 120

Manual desde cero para compilar, ejecutar, mover y defender el gemelo digital con pinza paralela.

RESULTADO VERIFICADO	PRECISIÓN
9 pruebas aprobadas	$2.22 \times 10^{-16} \text{ m}$

Versión comprobada el 10 de septiembre de 2026 · Ubuntu 24.04 · ROS 2 Jazzy



0 Cómo usar este manual

Resultado actual

El Hito 1 está completo en su parte cinemática y funcional. El CAD real queda pendiente, pero hoy el robot se puede compilar, visualizar, mover y validar con geometría provisional.

Lee primero los capítulos 1 a 4 para entender las ideas. Para la demostración en vivo usa los capítulos 8 a 11. El capítulo 15 contiene respuestas cortas para la defensa.

No necesitas memorizar todos los comandos. Comprende qué prepara cada uno y usa la hoja rápida del final como apoyo.

Contenido

Cap.	Tema
1	Qué exigía el Hito 1
2	Conceptos mínimos de ROS 2
3	Qué son URDF y Xacro
4	Cómo se construyó el robot
5	Modelo Denavit-Hartenberg
6	Arquitectura del paquete
7	Qué se creó, modificó y eliminó
8	Entrar a ROS 2 y compilar manualmente
9	Abrir RViz y mover automáticamente
10	Mover el robot manualmente
11	Inspeccionar nodos, tópicos, TF y error
12	Pruebas y evidencia
13	Errores comunes
14	Formato CAD que debes conseguir
15	Preguntas probables de defensa
16	Guion de demostración de 7 minutos
17	Hoja rápida de comandos

1 Qué exigía el Hito 1

La Guía PFI solicita un paquete ROS del ABB IRB 120 con pinza, un modelo URDF modular y una demostración de que la pose cartesiana coincide con una cinemática directa implementada por el equipo. El error posicional debe ser menor a 1×10^{-6} m. No se debe delegar la solución cinemática a MoveIt.

Traducción a tareas concretas

- Crear un paquete ROS 2 que pueda compilarse con colcon.
- Describir base, seis eslabones, seis articulaciones y pinza en URDF/Xacro.
- Publicar estados articulares y transformaciones para que RViz dibuje el robot.
- Programar la cinemática directa DH explícitamente en Python y NumPy.
- Comparar DH contra una evaluación independiente de la cadena URDF.
- Mostrar evidencia repetible: pruebas, error numérico y demostración visual.

Criterio de aprobación

Error máximo medido: 2.220446049250e-16 m. Límite: 1.0e-6 m. El resultado es APROBADO.

Qué no está incluido todavía

La geometría exterior exacta del fabricante no está disponible. Por eso los cuerpos naranjas son cilindros, cajas y esferas provisionales. Esto cambia la apariencia, pero no modifica los frames, los ejes, las longitudes DH ni la posición de tool0.

2 Conceptos mínimos de ROS 2

Concepto	Explicación sencilla	Ejemplo del proyecto
ROS 2	Infraestructura que permite comunicar programas robóticos.	ROS 2 Jazzy
Workspace	Carpeta de trabajo que agrupa paquetes fuente y resultados de compilación.	ros2_ws
Paquete	Unidad organizada de código, recursos y metadatos.	irb120_hito1
Nodo	Programa que realiza una función concreta.	motion_demo
Tópico	Canal por el que los nodos publican y reciben mensajes.	/joint_states
Mensaje	Estructura de datos que viaja por un tópico.	JointState
Frame	Sistema de coordenadas unido a una parte del robot.	base_link, tool0
TF	Sistema que mantiene transformaciones entre frames a lo largo del tiempo.	base_link a tool0
RViz	Visualizador 3D de datos ROS. No es un simulador dinámico.	Ventana del robot
Launch	Archivo que inicia varios nodos con una sola orden.	display.launch.py

La idea completa en una cadena

1. Comando	2. Estado	3. Transformaciones	4. Visualización
motion_demo o usuario	joint_state_source publica /joint_states	robot_state_publisher calcula TF desde URDF	RViz dibuja links y frames

Idea clave

ROS 2 no es el robot ni la matemática. Es la infraestructura que conecta la fuente de posiciones, el modelo, el validador y el visualizador.

3 Qué son URDF y Xacro

URDF significa Unified Robot Description Format. Es un documento XML que representa el robot como un árbol. Los nodos del árbol son links y las conexiones son joints.

Elementos de un link

- visual: forma y material que se ve en RViz.
- collision: forma simplificada que se usaría para detectar colisiones.
- inertial: masa, centro de masa y matriz de inercia.

Elementos de un joint

- parent y child: indican qué dos links conecta.
- origin: posición y orientación fija del joint respecto al padre.
- axis: eje alrededor del cual rota o se desplaza.
- limit: rango, velocidad y esfuerzo permitidos.
- type: fixed, revolute, continuous o prismatic, entre otros.

Xacro no reemplaza URDF: lo genera

URDF puro puede ser largo y repetitivo. Xacro agrega variables, propiedades y macros. En este proyecto un macro crea el IRB 120 y otro crea la pinza. El archivo principal llama a ambos. Luego Xacro se expande a un URDF normal que ROS puede leer.

```
xacro irb120_with_gripper.urdf.xacro -o irb120_with_gripper.urdf
```

Defensa

Si preguntan por qué se entregan Xacro y URDF: Xacro demuestra modularidad; el URDF expandido permite inspección y validación reproducible.

4

Cómo se construyó el robot

La construcción se hizo por capas para que cada parte tenga una responsabilidad clara.

Pas o	Qué se hizo	Archivo principal
1	Se transcribió la tabla DH y se implementaron matrices homogéneas.	kinematics/dh.py
2	Cada fila DH se descompuso en un joint móvil y frames fijos URDF.	urdf/irb120.macro.xacro
3	Se añadieron visuales provisionales, colisiones, límites e inercias.	urdf/irb120.macro.xacro
4	Se creó una pinza paralela modular con dos dedos prismáticos y TCP.	urdf/gripper.macro.xacro
5	El archivo principal unió materiales, robot y pinza.	urdf/irb120_with_gripper.urdf.xacro
6	Se expandió Xacro y se creó un evaluador URDF independiente.	kinematics/urdf_chain.py
7	Se añadieron nodos de estados, movimiento y validación.	nodes/
8	Se creó un launch integral y una vista RViz preparada.	launch/ y rviz/
9	Se escribieron pruebas para DH, URDF y movimiento.	test/

Cómo se unió la pinza

El último frame del robot es tool0. Se añadió un joint fijo gripper_mount que conecta tool0 con gripper_base_link. Dos joints prismáticos desplazan los dedos en sentidos opuestos. Finalmente, tcp_link se fija a 0.160 m desde tool0 como valor provisional.

Modularidad

Cuando llegue el CAD, se reemplazan las etiquetas visual y collision. No se cambian los joints ni la matemática validada.

5

Modelo Denavit-Hartenberg

Se usa DH clásico. La transformación de un frame al siguiente es:

$$T(i-1,i) = R_z(\theta_i) \cdot T_z(d_i) \cdot T_x(a_i) \cdot R_x(\alpha_i)$$

Eje	theta efectivo	d [m]	a [m]	alpha [rad]
1	q1	0.290	0.000	-pi/2
2	q2 - pi/2	0.000	0.270	0
3	q3	0.000	0.070	-pi/2
4	q4	0.302	0.000	pi/2
5	q5	0.000	0.000	-pi/2
6	q6 + pi	0.072	0.000	0

Qué significa cada parámetro

- theta: rotación alrededor del eje Z anterior. En un robot revoluto contiene q.
- d: desplazamiento a lo largo del eje Z anterior.
- a: longitud común normal entre dos ejes, medida sobre X.
- alpha: giro entre los ejes Z, medido alrededor de X.

La cinemática directa multiplica seis matrices. El resultado es una matriz homogénea 4 x 4. Su bloque superior izquierdo 3 x 3 representa orientación y la última columna representa la posición [x, y, z].

Por qué URDF y DH pueden coincidir exactamente

URDF aplica primero el origen fijo y después el movimiento del joint. Para respetar el orden DH, cada fila se representa como $R_z(q)$ en el joint móvil, luego un frame fijo con el offset angular y otro frame fijo con $T_z(d)$, $T_x(a)$ y $R_x(\alpha)$.

6

Arquitectura del paquete

```
irb120_hito1/
├─ package.xml           dependencias ROS
├─ setup.py              instalación y ejecutables
├─ launch/display.launch.py  inicia todo el sistema
├─ config/               parámetros articulares
├─ rviz/hito1.rviz       vista 3D preparada
├─ urdf/
│   ├── materials.xacro
│   ├── irb120.macro.xacro
│   ├── gripper.macro.xacro
│   ├── irb120_with_gripper.urdf.xacro
│   └── irb120_with_gripper.urdf
├─ irb120_hito1/
│   ├── kinematics/dh.py
│   ├── kinematics/urdf_chain.py
│   ├── motion.py
│   ├── nodes/joint_state_source.py
│   ├── nodes/motion_demo.py
│   ├── nodes/fk_validator.py
│   └── verify_model.py
└─ test/
    ├── test_dh.py
    ├── test_motion.py
    └── test_urdf_matches_dh.py
```

Ejecutable	Entrada	Salida / función
joint_state_source	Comandos en grados	Publica /joint_states y aplica límites
motion_demo	Tiempo	Publica una secuencia suave y cíclica
robot_state_publisher	URDF + /joint_states	Publica TF
fk_validator	/joint_states	Compara DH contra URDF y publica error
verify_model	URDF + posturas fijas	Pasaporte offline reproducible
rviz2	URDF + TF	Muestra robot, grilla y frames

7

Qué se creó, modificó y eliminó

Creado para el Hito 1

- Paquete ROS 2 irb120_hito1 con recursos instalables.
- Modelo Xacro modular y URDF expandido.
- Implementación DH y evaluador de cadena URDF independientes.
- Nodos para estados articulares, validación y movimiento automático.
- Configuración RViz, archivo launch y nueve pruebas.
- Documentos de modelo, trazabilidad, resultados, guion, manual y presentación.
- Capturas reales del robot funcionando en RViz.

Modificado o mejorado

- README principal y README del paquete con comandos reales.
- Parámetros DH, límites oficiales, masas e inercias aproximadas documentadas.
- Validador en vivo ajustado para comparar la misma muestra por DH y URDF durante movimiento continuo.
- Launch ampliado con el argumento demo_motion.
- .gitignore ampliado para evitar resultados de compilación en la raíz.

Eliminado

Se movieron a la papelera las carpetas build, install y log de la raíz. Eran resultados regenerables creados por compilar desde la carpeta equivocada. No se eliminó código fuente ni la compilación correcta de ros2_ws.

Qué falta

Solo falta integrar el CAD real y confirmar datos finales de la pinza. Para la entrega de hoy, la función y la precisión del Hito 1 están completas.

8 Entrar a ROS 2 y compilar manualmente

Regla de oro

Cada terminal nueva empieza sin conocer ROS ni tu workspace. Debes cargar ambos entornos con source.

Paso 1. Abrir una terminal

Presiona Ctrl+Alt+T. El símbolo \$ indica que la terminal está esperando una orden. Escribe una línea y presiona Enter.

Paso 2. Entrar al proyecto

```
cd /home/marcelo767/Documentos/robotica/robotica-IMT342/2-2026/IMT342/equipo/proyecto
pwd
ls
```

pwd debe terminar en /proyecto. ls debe mostrar ros2_ws, docs, documentosGuia y otros archivos.

Paso 3. Cargar ROS 2 Jazzy

```
source /opt/ros/jazzy/setup.bash
ros2 --help
```

source modifica solo la terminal actual. No instala nada y no cambia archivos.

Paso 4. Entrar al workspace y compilar

```
cd ros2_ws
colcon build --symlink-install --packages-select irb120_hito1
```

src contiene el código. build guarda trabajo temporal. install contiene lo ejecutable. log guarda reportes. Debes compilar desde ros2_ws, no desde proyecto.

Paso 5. Cargar lo recién compilado

```
source install/setup.bash
ros2 pkg prefix irb120_hito1
```

El último comando debe mostrar una ruta dentro de ros2_ws/install/irb120_hito1. Si dice que no existe, faltó compilar o cargar install/setup.bash.

Miniconda

En esta computadora no ejecutes nodos ROS con el Python de Miniconda. Usa ros2 después de los dos source. Para diagnósticos directos usa /usr/bin/python3.

9

Abrir RViz y mover automáticamente

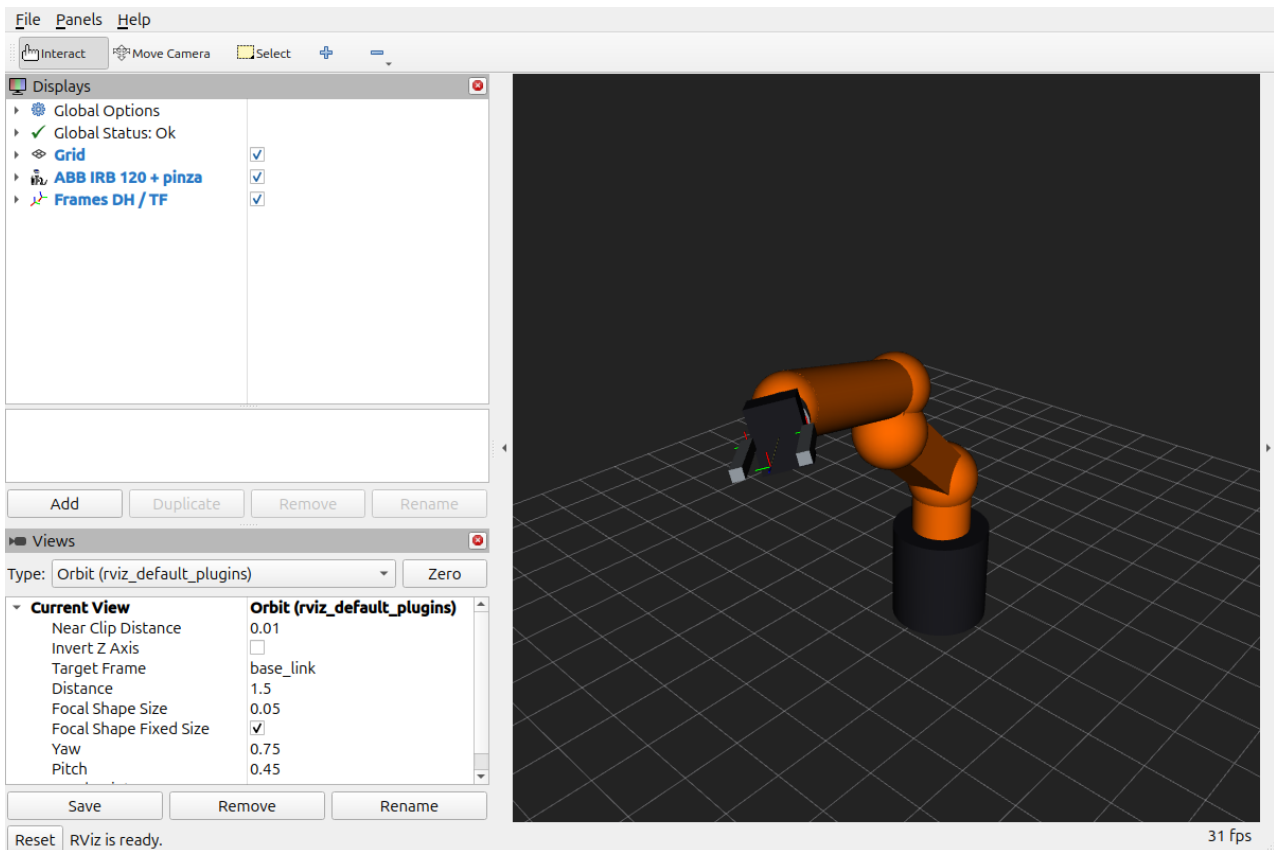
Con ROS y el workspace cargados, ejecuta:

```
ros2 launch irb120_hito1 display.launch.py demo_motion:=true
```

Este único comando inicia cinco procesos: robot_state_publisher, joint_state_source, fk_validator, motion_demo y rviz2.

Qué debes observar

- Se abre una ventana 3D de RViz.
- El brazo naranja cambia de postura suavemente.
- Los dos dedos de la pinza se abren y cierran.
- Los ejes de colores muestran los frames DH y TF.
- La terminal imprime APROBADO | DH vs URDF aproximadamente una vez por segundo.



Captura real de la demostración automática en esta computadora. La forma exterior es provisional; la cadena cinemática es la validada.

Para terminar

Vuelve a la terminal donde ejecutaste launch y presiona Ctrl+C una sola vez. ROS cerrará todos los procesos.

10 Mover el robot manualmente

Usa dos terminales. La primera mantiene el sistema encendido. La segunda envía comandos.

Terminal 1

```
source /opt/ros/jazzy/setup.bash
cd /home/marcelo767/Documentos/robotica/robotica-IMT342/2-2026/IMT342/equipo/proyecto/ros2_ws
source install/setup.bash
ros2 launch irb120_hito1 display.launch.py
```

Terminal 2

```
source /opt/ros/jazzy/setup.bash
cd /home/marcelo767/Documentos/robotica/robotica-IMT342/2-2026/IMT342/equipo/proyecto/ros2_ws
source install/setup.bash
ros2 topic pub --once /irb120/joint_commands_deg std_msgs/msg/Float64MultiArray \
+ "{data: [30.0, -20.0, 15.0, 40.0, -35.0, 60.0, 0.010]}"
```

Interpretación del mensaje

Posición	Significado	Unidad
1 a 6	Ángulos q1 a q6	grados
7 opcional	Apertura de cada dedo	metros, 0 a 0.015

Prueba otras posturas seguras

```
# Postura cero
ros2 topic pub --once /irb120/joint_commands_deg std_msgs/msg/Float64MultiArray \
+ "{data: [0, 0, 0, 0, 0, 0, 0.0]}"

# Postura alternativa
ros2 topic pub --once /irb120/joint_commands_deg std_msgs/msg/Float64MultiArray \
+ "{data: [-45, 25, -30, 50, 20, -90, 0.015]}"
```

Límites

El nodo limita automáticamente valores fuera del rango oficial y muestra una advertencia. Esta protección es para el gemelo, no sustituye la seguridad de un robot físico.

11 Inspeccionar nodos, tópicos, TF y error

Qué quieres ver	Comando
Nodos activos	<code>ros2 node list</code>
Tópicos activos	<code>ros2 topic list</code>
Estados articulares	<code>ros2 topic echo /joint_states --once</code>
Error cinemático	<code>ros2 topic echo /irb120/fk_position_error_m</code>
Frecuencia de estados	<code>ros2 topic hz /joint_states</code>
Información del tópico	<code>ros2 topic info /joint_states -v</code>
Transformación base-herramienta	<code>ros2 run tf2_ros tf2_echo base_link tool0</code>
Argumentos del launch	<code>ros2 launch irb120_hito1 display.launch.py --show-args</code>

Cómo leer una transformación

`tf2_echo` muestra Translation [x, y, z] en metros y Rotation como cuaternión y RPY. `base_link` es el origen fijo; `tool0` es la brida del eje 6; `tcp_link` es el punto de trabajo de la pinza.

Para validar el TCP directamente:

```
ros2 launch irb120_hito1 display.launch.py target_frame:=tcp_link
```

Ctrl+C

Los comandos que escuchan continuamente, como `topic echo` y `tf2_echo`, se detienen con Ctrl+C.

12 Pruebas y evidencia

Pruebas automatizadas

```
colcon test --packages-select irb120_hito1 --event-handlers console_direct+
colcon test-result --verbose
```

Resultado esperado: 9 tests, 0 errors, 0 failures, 0 skipped.

Pasaporte offline

```
ros2 run irb120_hito1 verify_model
```

Métrica	Resultado	Criterio
Error posicional máximo	2.220446049250e-16 m	menor a 1.0e-6 m
Error de orientación máximo	1.004007835075e-15 rad	informativo
Posturas documentadas	4 x 2 frames	tool0 y tcp_link
Posturas aleatorias en tests	100 reproducibles	dentro de límites

Por qué hay un número pequeño y no cero

Los cálculos usan punto flotante. Diferencias alrededor de 1e-16 son redondeo numérico de la computadora, no un desplazamiento físico medible.

Dos caminos independientes

Camino A multiplica las matrices DH escritas en Python. Camino B recorre los joints del URDF expandido, interpreta origin, axis y tipo de joint, y multiplica sus transformaciones. Si ambos llegan a la misma pose en muchas posturas, los parámetros y su traducción a URDF son coherentes.

13 Errores comunes

Síntoma	Causa probable	Solución
ros2: command not found	No se cargó ROS.	<code>source /opt/ros/jazzy/setup.bash</code>
Package irb120_hito1 not found	No se cargó el workspace o no se compiló.	Compila desde <code>ros2_ws</code> y ejecuta <code>source install/setup.bash</code>
No aparece RViz	Launch se ejecutó con <code>use_rviz:=false</code> o falló el entorno gráfico.	Usa el comando normal y revisa la terminal
El robot no se mueve	No está activo <code>motion_demo</code> y no llegó un comando.	Usa <code>demo_motion:=true</code> o publica la postura manual
El comando manual no funciona	La segunda terminal no tiene los <code>source</code> .	Carga ROS y <code>ros2_ws/install</code> en esa terminal
Error de <code>rcipy</code> o <code>libstdc++</code>	Se mezcló Miniconda con ROS Jazzy.	Abre una terminal normal; evita python de Conda
Se crearon <code>build/install/log</code> en proyecto	<code>colcon</code> se ejecutó una carpeta arriba.	Elimina esos generados y compila dentro de <code>ros2_ws</code>
RViz muestra geometría simple	Aún no se integró el CAD.	Es el estado provisional previsto; la cinemática sí es final

Diagnóstico mínimo

```
pwd
which ros2
ros2 pkg prefix irb120_hito1
ros2 node list
ros2 topic list
```

Contraseñas

Nunca escribas una contraseña dentro de un script, README, historial compartido o presentación. `sudo` solo debe pedirla de forma interactiva si realmente falta instalar algo.

14 Formato CAD que debes conseguir

Opción ideal

Un paquete URDF/Xacro oficial o confiable con un mesh separado para base_link y cada link_1 a link_6. Para visual conviene Collada .dae porque conserva colores y materiales. Para colisión conviene .stl simplificado porque es compatible y ligero.

Opción aceptable

Un ensamble STEP .step o .stp con cada eslabón como componente independiente. Debe incluir unidades, postura cero, nombres de piezas y ubicación de los ejes. Después se exportará cada eslabón por separado.

Formato	Uso	Recomendación
.dae	Visual en RViz	Preferido si conserva colores
.stl	Visual o colisión	Usar uno simplificado para colisión
.step / .stp	Fuente CAD editable	Aceptar si el ensamble separa los eslabones
.obj	Visual	Posible, pero menos común en paquetes ROS
Una sola malla fusionada	Ningún uso cinemático	No sirve para mover eslabones por separado

Información adicional necesaria

- Unidades exactas del archivo, preferentemente metros o milímetros claramente indicados.
- Postura de calibración $q = [0,0,0,0,0,0]$.
- Origen y eje de giro de cada articulación.
- Para la pinza: carrera, distancia tool0-TCP, masa, centro de masa y orientación de montaje.

No descargues una sola pieza

Para un gemelo articulado, cada eslabón debe poder transformarse de forma independiente.

15 Preguntas probables de defensa

Pregunta	Respuesta breve
¿Qué es ROS 2?	Infraestructura para que procesos robóticos intercambien datos y coordinen funciones.
¿Qué es URDF?	XML que describe links, joints, geometría, límites y propiedades físicas de un robot.
¿Qué aporta Xacro?	Macros, propiedades y reutilización; genera el URDF final sin duplicar XML.
¿Qué diferencia hay entre link y joint?	Un link es un cuerpo rígido; un joint define cómo se conecta y mueve respecto a otro.
¿Qué es TF?	Sistema de ROS que mantiene la relación espacial entre frames a lo largo del tiempo.
¿RViz simula física?	No. Visualiza datos y transformaciones; no calcula contacto, fuerza ni dinámica.
¿Qué es DH?	Convención de cuatro parámetros para encadenar transformaciones entre eslabones.
¿Cómo se validó?	Se comparó el producto DH con un recorrido independiente de la cadena URDF.
¿Por qué usar varias posturas?	Una sola postura puede ocultar errores de signo, offset o eje.
¿Se usó MoveIt?	No. La cinemática y la evaluación URDF fueron programadas explícitamente.
¿Qué significa 1e-6 m?	Un micrómetro: una millonésima de metro.
¿Por qué falta el CAD?	El CAD solo sustituirá visuales y colisiones; la cadena matemática ya está cerrada.
¿Controla un ABB real?	No todavía. Este hito valida el gemelo cinemático, no la integración segura con hardware.

16 Guion de demostración de 7 minutos

Tiempo	Qué muestras	Qué dices
0:00-1:00	Diapositiva de objetivo	Paquete ROS, URDF modular, pinza y error menor a $1e-6$ m.
1:00-2:00	Estructura del paquete	Separé matemática, URDF, nodos, launch, configuración y pruebas.
2:00-3:00	Tabla DH y Xacro	Cada fila DH se descompuso sin cambiar el orden de transformaciones.
3:00-4:00	Flujo ROS	Comando, JointState, TF, RViz y validación.
4:00-5:30	RViz en movimiento	Seis ejes y pinza; la terminal permanece en APROBADO.
5:30-6:20	verify_model	Muestro $2.22e-16$ m frente al límite $1e-6$ m.
6:20-7:00	Pendiente CAD	La apariencia es provisional; los frames y la cinemática son finales.

Comandos que conviene dejar listos

```
# Primero, evidencia numérica
ros2 run irb120_hito1 verify_model

# Después, demostración visual
ros2 launch irb120_hito1 display.launch.py demo_motion:=true
```

Si RViz demora

No repitas el comando. Espera unos segundos y mira si la terminal informa que rviz2 inició. Si falla, muestra la captura incluida en la presentación y ejecuta verify_model.

17 Hoja rápida de comandos

Inicio completo desde una terminal nueva

```
cd /home/marcelo767/Documentos/robotica/robotica-IMT342/2-2026/IMT342/equipo/proyecto
source /opt/ros/jazzy/setup.bash
cd ros2_ws
colcon build --symlink-install --packages-select irb120_hito1
source install/setup.bash
```

Validar

```
colcon test --packages-select irb120_hito1 --event-handlers console_direct+
colcon test-result --verbose
ros2 run irb120_hito1 verify_model
```

Demostrar

```
ros2 launch irb120_hito1 display.launch.py demo_motion:=true
```

Mover manualmente

```
ros2 topic pub --once /irb120/joint_commands_deg std_msgs/msg/Float64MultiArray \
+ "{data: [30, -20, 15, 40, -35, 60, 0.010]}"
```

Inspeccionar

```
ros2 node list
ros2 topic list
ros2 topic echo /irb120/fk_position_error_m
ros2 run tf2_ros tf2_echo base_link tool0
```

Secuencia mental

Entrar al proyecto. Cargar ROS. Entrar a ros2_ws. Compilar. Cargar install. Ejecutar. Ctrl+C para terminar.

18 Fuentes y archivos de referencia

Fuente	Uso en el proyecto
Guia_PFI.pdf, sección 4	Requisitos y pasaporte del Hito 1
Bahani et al., IJMERR 2023	Tabla DH del ABB IRB 120
Truc y Lam, 2020	Masas, centros de masa e inercias CAD aproximadas
ABB Product specification IRB 120, 3HAC035960-001 Rev. S	Rangos articulares y velocidades máximas
docs/hito1/modelo_cinematico.md	Convención, tabla y decisiones técnicas
docs/hito1/resultados.md	Registro de pruebas y mediciones reales
docs/hito1/trazabilidad.md	Relación requisito-implementación-verificación

Ruta del proyecto

```
/home/marcelo767/Documentos/robotica/robotica-IMT342/2-2026/IMT342/equipo/proyecto
```

Este manual fue generado a partir de la versión realmente compilada y probada en esa ruta el 10 de septiembre de 2026.